

SumTypes and Ruby

Ruby Melbourne

Sid 27/08/2026

SumTypes

- What is?
- History & Other languages
- in Ruby
 - Existing solutions
 - Build your own

STI

Ruby on Rails 8.1.3.1

Module

ActiveRecord::Inheritance

activerecord/lib/active_record/inheritance.rb

Single table inheritance ↱ ↑

Active Record allows inheritance by storing the name of the class in a column (e.g. `Base.inheritance_column`). This means that an inheritance looking

```
class Company < ActiveRecord::Base; end
class Firm < Company; end
class Client < Company; end
class PriorityClient < Client; end
```

Single Table Inheritance

- Base Model has common attributes
- SubClasses have specific attributes
- DB Table always has
 - *all* the attributes of *all* the subclasses
 - + a “type” attribute (or a “tag”) to denote the “SubClass”
 - Most of the columns will be NULL

Single Table Inheritance

Jobs Table

id	type	queued_at	started_at	result	error
1	Queued	09:00	NULL	NULL	NULL
2	Running	NULL	09:12	NULL	NULL
3	Done	NULL	NULL	"42 rows"	NULL
4	Failed	NULL	NULL	NULL	"disk full"

STI BaseModel = Product Type

Sum Type $\neq$ Product Type

“OR”

“one of these”

“AND”

“all of these”

SumTypes Characteristics

- Closed, known set of variants
- Variants carry their own data
- Variant data is *usually* immutable
- Exhaustive checking / pattern matching

All four, in one place

```
enum JobStatus {
    Queued,
    Running { started_at: String },
    Done    { result: String },
}

fn describe(status: &JobStatus) -> String {
    match status {
        JobStatus::Queued                => "waiting to start".into(),
        JobStatus::Running { started_at } => format!("running since {started_at}"),
        JobStatus::Done { result }        => format!("finished: {result}"),
    }
}
```

Add a variant, forget the branch

```
enum JobStatus {  
    Queued,  
    Running { started_at: String },  
    Done     { result: String },  
    Failed   { error: String },    // ← new  
}
```

```
error[E0004]: non-exhaustive patterns:  
    `&JobStatus::Failed { .. }` not covered  
--> job_status.rs:17:11
```


SumTypes: in the wild

- 1965: Tony Hoare: NULL + SumType-style modelling
- 1970s: Robin Milner: ML (no data types though)
- 1980: HOPE (research language - first complete package)
- 1983: Standard ML: SumTypes take their final form
- 1990s: Haskell, OCaml
- 2000s: Scala, F#
- 2010s: Rust, Swift, Elm, Kotlin, Odin, Gleam, TypeScript, Python, Java

Ruby 🥰

Recap: SumTypes Characteristics

- ✓ • Closed, known set of variants
- ✓ • Variants carry their own data
- ✓ • Variant data is *usually* immutable
- ⚠ • Exhaustive checking / pattern matching

Ruby fails two ways

```
case job_status
in Queued          then "waiting"
in Running(started_at:) then "started #{started_at}"
in Done(result:)   then "finished: #{result}"
end
```

NoMatchingPatternError

The one people actually write

```
case job_status
in Queued          then "waiting"
in Running(started_at:) then "started #{started_at}"
in Done(result:)   then "finished: #{result}"
in _               then "unknown"          # ← the catch-all
end
```

```
=> "unknown"
```

Silent. Wrong. Ships.

Reach for a gem: dry-matcher

```
Describe = Dry::Matcher.new(  
  queued: Case.new(match: ->(v) { v.is_a?(Queued) }),  
  running: Case.new(match: ->(v) { v.is_a?(Running) }),  
  done: Case.new(match: ->(v) { v.is_a?(Done) })  
)  
  
def self.describe(status)  
  Describe.call(status) do |m|  
    m.queued { "waiting to start" }  
    m.running { |s| "running since #{s.started_at}" }  
    m.done { |s| "finished: #{s.result}" }  
  end  
end
```

Add **Failed**. Run it.

```
the NEW state (Failed), for which no case was ever declared:  
=> nil
```

nil. Not an error — a value. The exhaustiveness checker was **satisfied**.

What it *can* catch

```
a declared case left unhandled:  
=> Dry::Matcher::NonExhaustiveMatchError: cases +done+ not handled
```

Caught — at **call time**, on the one invocation that ran.

DIY

Code -> Data

```
case status          # ← CODE. opaque.
in Queued then ...   #   you cannot ask it what it covers
end

JobStatus.matcher(   # ← DATA. a hash.
  queued: ->(s) { ... },      #   you CAN ask it what it covers
  running: ->(s) { ... },
)
```

case/in is code. A handler hash is **data**.

40 lines of plain Ruby

```
module SumType
  def variants(**map) = @variants = map.freeze
  def variants_map    = @variants

  def matcher(**handlers)
    missing = variants_map.keys - handlers.keys
    raise NonExhaustiveMatch, "missing handlers for: #{missing.join(', ')}" if missing.any?
    Matcher.new(sum: self, handlers: handlers.freeze)
  end

  Matcher = Data.define(:sum, :handlers) do
    def call(value)
      tag = sum.variants_map.find { |_, type| value.is_a?(type) }&.first
      handlers.fetch(tag).call(value)
    end
  end
end
```

No gem. No dependency. It raises when the matcher is built.

Using it

```
module JobStatus
  extend SumType

  Queued   = Data.define
  Running  = Data.define(:started_at)
  Done     = Data.define(:result)

  variants queued: Queued, running: Running, done: Done
end

# a CONSTANT – this matters in about 30 seconds
Describe = JobStatus.matcher(
  queued:  ->(_) { "waiting to start" },
  running: ->(s) { "running since #{s.started_at}" },
  done:    ->(s) { "finished: #{s.result}" },
)
```


The same commit as before

```
variants(  
  queued: Queued,  
  running: Running,  
  done: Done,  
  failed: Failed,      # ← jobs can fail. add the variant.  
)
```

Describe is **not** updated.

bin/rails

```
$ bin/rails runner 'nil'
```

```
app/models/concerns/sum_type.rb:36:in 'SumType#matcher':  
  missing handlers for: failed (SumType::NonExhaustiveMatch)  
    from app/models/job_status_ops.rb:12:in '<module:JobStatusOps>'
```

```
BOOT FAILED (exit 1)
```


Tradeoff: when do you find out?

	When	Can you forget?	Silent wrong answer?
<code>case/in</code> + <code>else</code>	runtime, prod	—	yes
<code>case/in</code> , no <code>else</code>	runtime, prod	—	no
tests	CI	yes — write it	yes
Sorbet <code>T.absurd</code>	edit-time	yes — write it	no
<code>SumType</code>	boot	no	no
Rust / Gleam / Scala	edit-time	no	no

DIY Ruby SumType Module

<https://gist.github.com/sidbhatt11/5538e2e7d47efa3cddb16ace8556d42a>

```
60 module SumType
61   class Error < StandardError; end
62   class NonExhaustiveMatch < Error; end
63   class UnknownVariant < Error; end
64   class UnrecognisedValue < Error; end
65
66   # this builds the variants map/registry exactly once
67   def variants(**map)
68     raise Error, 'variants already declared' if defined?(@variants) && @variants
69     @variants = map.freeze
70   end
71
72   def variants_map
73     @variants || raise(Error, "#{self} declares no variants – call `variants(tag: SomeClass, ...)` first")
74   end
75
76   # added for convenience/interrogation
77   def variant?(klass) = variants_map.values.include?(klass)      # asks about a class
78   def member?(value) = variants_map.values.any? { value.is_a?(it) } # asks about a value
79
80   # this is what raises at construction time
81   def matcher(**handlers)
82     keys      = variants_map.keys
83     missing = keys - handlers.keys
```



